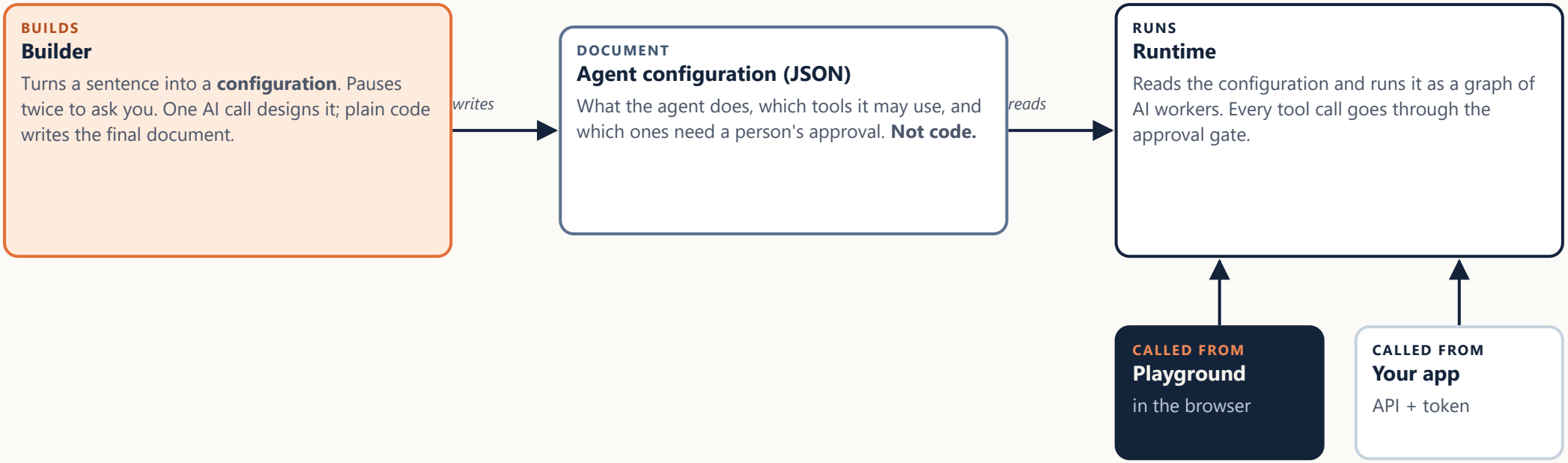
Forge: how it works

Two parts — one builds agents, one runs them — sharing a registry, a vault and a database.



Shared foundations — used by both the builder and the runtime

TOOLS
MCP Registry
The apps a company has plugged in (GitHub, Slack, Jira, files…) and every tool they offer, each labelled **read**, **write** or **destructive**.

LOGINS
Vault
Each person's app logins, encrypted. Lent out for one tool call at a time, never stored in an agent.

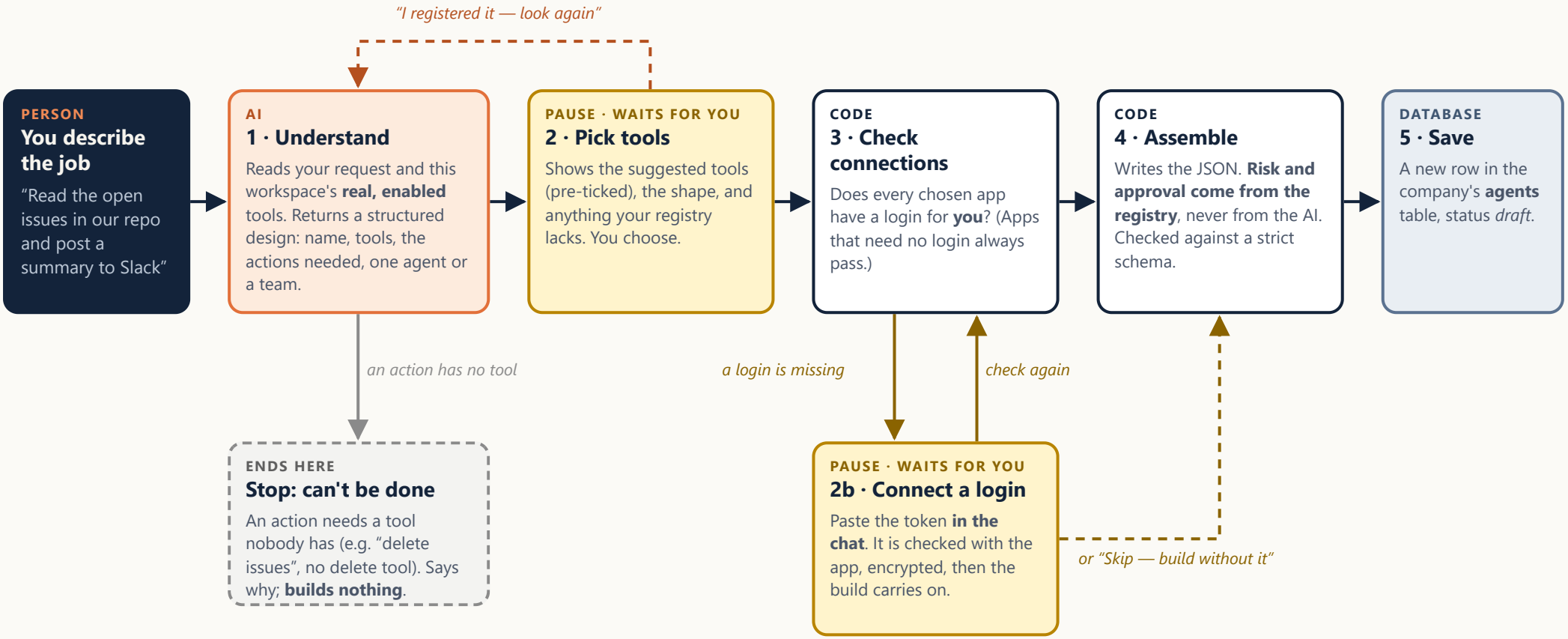
DATA
Postgres, one sealed space per company
Agents, runs and connections per company. Every pause is saved here, so it survives a restart.

The one idea everything hangs off: an agent is a configuration document, not generated code. The builder never writes Python, and one runtime runs every agent, so the safety rules live in one place and apply to all of them.

The builder, as built

Five steps, **one AI call**, two places where it stops and waits for you.

Person · AI step · Plain code · Pause (waits for a person) · Database · Ends here



Every pause is saved

Both pauses are LangGraph interrupts stored in the company's own schema (checkpoints). Close the browser or restart the server: the same question is waiting when you come back.

The configuration

A real example: the document the builder saves and the runtime reads.

```
{
  "schema_version": "1.0",
  "name": "Issue Digest",
  "description": "Triages open GitHub issues and posts a digest to Slack.",
  "model": { "provider": "google_genai",
             "name": "gemini-flash-lite-latest", "temperature": 0 },
  "topology": {
    "type": "supervisor",
    "supervisor": {
      "instructions": "First send the work to triager, then the
                       triaged list to reporter. Do not call tools.",
      "delegates_to": ["triager", "reporter"] },
    "specialists": [
      { "name": "triager",
        "instructions": "List the open issues and classify each P0-P2.",
        "tools": ["github.list_issues", "github.get_issue"] },
      { "name": "reporter",
        "instructions": "Write a Slack digest under 200 words, P0s first.",
        "tools": ["slack.slack_send_message"] } ] },
  "tools": [
    { "ref": "github.list_issues", "risk": "read",
      "approval": "auto", "requires_connection": "github" },
    { "ref": "github.get_issue", "risk": "read",
      "approval": "auto", "requires_connection": "github" },
    { "ref": "slack.slack_send_message", "risk": "write",
      "approval": "ask",  "requires_connection": "slack" } ],
  "policy": { "approval_required_for": ["write", "destructive"],
              "max_tool_calls": 25 },
  "requires_connections": ["github", "slack"],
  "schedule": null
}
```

Topology AI PROPOSES

Who does what. **supervisor** = a coordinator that hands work to specialists; **single** = one worker. Each specialist holds only its own tools.

Tools CODE FILLS IN, FROM THE REGISTRY

Every tool, with its **risk** and whether it must **ask** first. Write and destructive always ask.

Policy FIXED BY THE PLATFORM

Approval is required for write and destructive tools; at most 25 tool calls.

Requires connections CODE DERIVES IT

Which apps each person must connect before running it.

Schedule NOT USED YET

Always **null** today: agents run when someone asks. Timed runs are not built yet.

The AI never decides what is risky. It proposes names, instructions and which tools to use; plain code copies risk and approval from the registry and validates the whole document against a strict schema before it is saved.

The runtime

Turns a configuration into a running graph, then runs it.

① Compile — fresh on every run

DOCUMENT
Configuration
Loaded from the company's **agents** table.

→

CODE
Drop switched-off tools
Anything an admin turned off since the agent was built is left out.

→

CODE
Ask each app for its tools
The real argument lists come live from each server, never typed by hand.

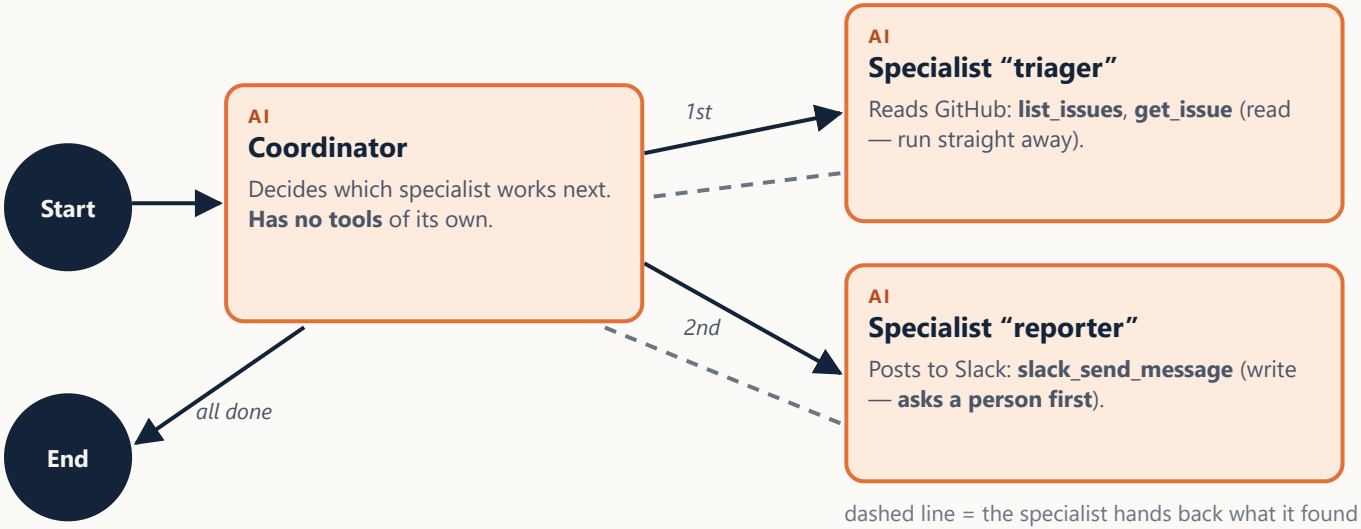
→

CODE
Wrap every tool
Each one is put behind the **approval gate**.

→

CODE
Build the graph
One worker, or a coordinator plus specialists.

② Run — the coordinator shape



dashed line = the specialist hands back what it found

Inside every worker

The AI calls a tool, reads the result, and calls again: **up to 6 rounds**, then it must answer. If it runs out, it is asked once (tools taken away) to say what it found.

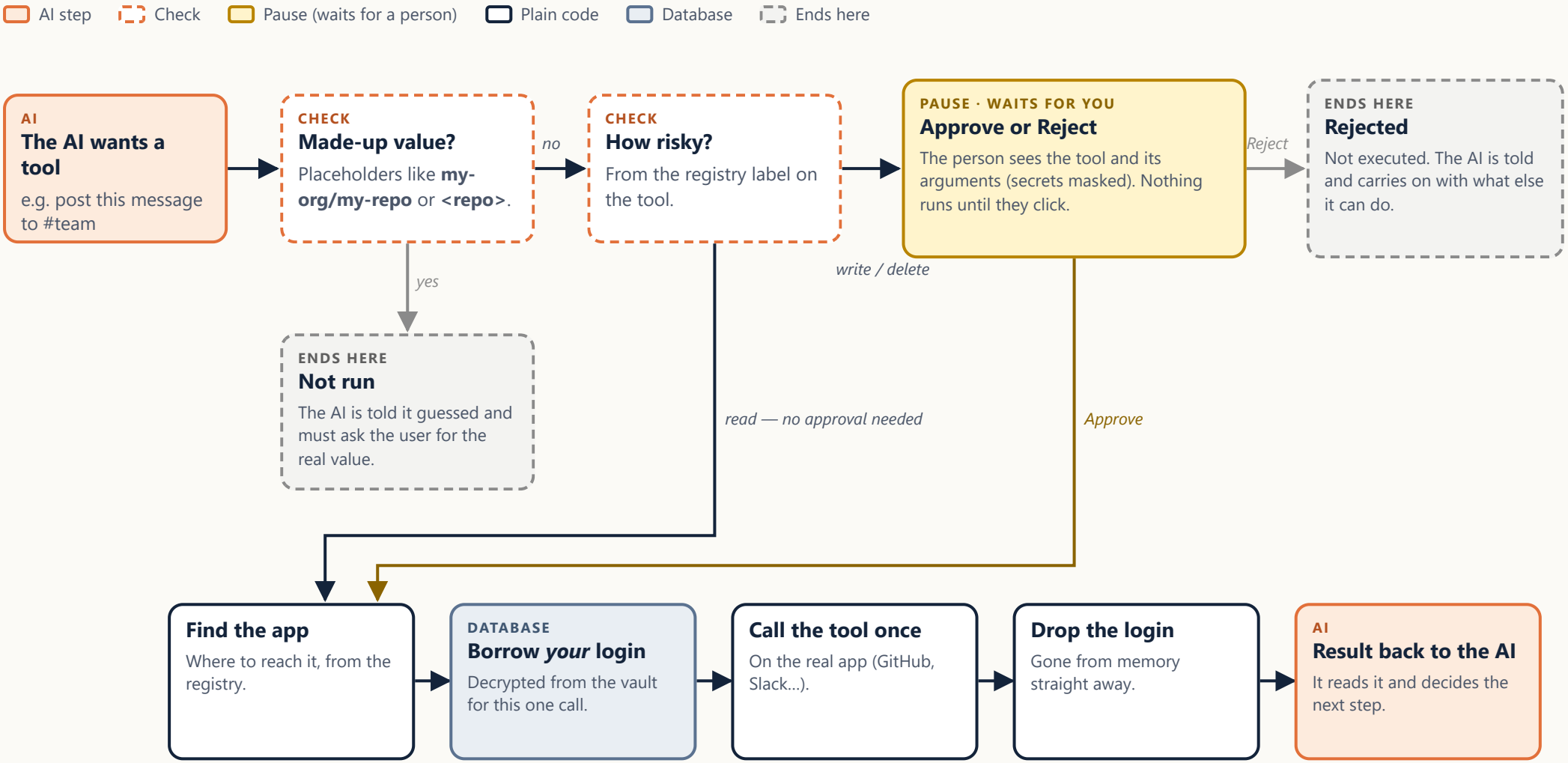
It is told never to invent a value it was not given (a repository, a channel), and to ask instead.

Single-agent shape: Start → one worker → End, same loop.

Because the graph is compiled from the configuration on every run, an admin's change to the registry (a tool switched off) applies immediately, even to agents built weeks ago and to runs resumed after a pause.

The approval gate

Every single tool call goes through these steps, for every agent.



This gate is in the platform, not in the AI's instructions: a write tool physically cannot run before a person answers. The pause is saved, so it survives a server restart and can wait days.

Invoking an agent

Two ways in today; both go through the same runtime and the same approval gate.

Who can start a run

BUILT

Playground

In the browser, signed in. Shows each step live as it happens (streamed).

BUILT

Your app or script

With an API token (**forge_...**): PowerShell, curl or the Postman collection from the API tab. Another app = this route.

What happens on every call

1 · Who is calling?

Session cookie or API token → this person and their company.

2 · Load the agent

From that company's sealed space. Another company's id → **Not found**.

3 · Check logins

The Playground asks you to connect first; the API runs and reports any gap.

4 · Compile and run

Configuration → graph → run, saved under a thread id.

5 · Record the run

Status, steps, time taken — in the company's **runs** table.

PAUSE · WAITS FOR YOU

If it reaches a write tool

The run stops with status **awaiting approval**. Answer with Approve / Reject in the Playground, or **POST .../runs/{run}/resume** from your app.

Endpoints

POST /v1/agents/{id}/invoke
POST /v1/agents/{id}/stream
POST /v1/agents/{id}/runs/{run}/resume
GET /v1/agents/{id}/runs

Not built yet (in the first design)

NOT BUILT YET

On a timetable

e.g. every morning at 9

NOT BUILT YET

On an event

e.g. when an issue is opened

NOT BUILT YET

Agent to agent

one agent calling another

Where things are stored

One Postgres database: a shared schema, plus one sealed schema per company.

`platform` shared by all companies

`users`	everyone's login: email, password hash (one-way), role, company
`tenants`	one row per company and its schema name
`mcp_servers / mcp_tools`	apps the platform admin shared with every company
`listings`	the public marketplace
`submission_index`	publish requests waiting for review
`api_tokens`	a **hash** of each forge_ token, never the token

`t_<company>` one sealed copy per company

`agents`	each agent's configuration, scores and status
`runs`	every run: input, steps, output, feedback
`connections`	each person's app logins, **encrypted**
`mcp_servers / mcp_tools`	apps registered privately or for the company, tools with risk + on/off
`submissions`	this company's publish attempts and the reviewer's notes
`checkpoint tables`	every paused build, run and review — why pauses survive restarts

Sealed per company. Every request switches to that company's own database role, which can only open its own schema. Another company's tables are off-limits to the database itself.

Private per person. Row-level security means you see only your own agents, runs and logins, plus apps your admin shared.

Not in the database: the master key that unlocks the vault (FORGE_MASTER_KEY, kept in the environment or a secrets manager).

First design vs as built

What changed between the original diagram and the working product.

Topic	First design (rough diagram)	As built
Builder steps	12 nodes, several AI calls (analyse, design, generate, repair)	5 steps and **one** AI call; the rest is plain code
Validate and repair	AI fixes its own validation errors in a loop	Code fills risk and approval from the registry and validates against a strict schema, so there is nothing for an AI to repair
User choice	Pick MCP servers	Pick individual tools (suggested ones pre-ticked); see what the registry lacks
Missing login	Ask the user, then resume	Same, and the token can be pasted **in the build chat**; it is checked first
Can't be done	—	The build refuses and says which action has no tool; nothing is saved
Format	YAML agent definition	JSON, validated by the AgentConfig schema
Versions	V1, V2… never overwritten	One agent per build; no version history yet
Deploy	Register, compile and cache the graph	No deploy step: the graph is compiled on every run, so registry changes apply at once
Logins	mcp_connections with a credential reference	connections holds the token itself, encrypted per person (envelope encryption)
Invocation	API, schedule, events, another app, UI, A2A	UI (Playground) and API; schedules, events and A2A not built